

Odoo Full-Stack Developer Program Training Plan

From Linux & Python Foundations to Production Odoo Deployment

Version Focus	Odoo 17 · Odoo 18 · Odoo 19
Audience	Aspiring & Intermediate Odoo Developers
Format	Instructor-led + Labs + Capstone Projects
Document Type	Master Training Plan & Syllabus
Edition	July 2026
Minimum Duration	120+ guided learning hours

Includes: Backend ORM · Views · Security · OWL · POS · APIs · DevOps · VPS

Companion document: Odoo Full-Stack Training Presentation.pdf

Table of Contents

Part A — Program Overview

1. Executive Summary & Program Vision
2. Who This Program Is For
3. Learning Journey Map
4. Program Structure & Duration Model
5. Assessment, Labs & Capstone Projects

Part B — Foundation Tracks

6. Module F1 — Linux OS & CLI Mastery
7. Module F2 — Python Programming for Odoo
8. Module F3 — PostgreSQL Administration

Part C — Odoo Core Development

9. Module O1 — Installation, CLI & Configuration
10. Module O2 — Custom Modules & Manifest
11. Module O3 — Fields & Model Design
12. Module O4 — ORM Methods Deep Dive
13. Module O5 — Views (List, Form, Search, Kanban...)
14. Module O6 — Security Architecture
15. Module O7 — Data Loading (XML / CSV)
16. Module O8 — View Inheritance & Migration Tips

Part D — Advanced Odoo

17. Module A1 — QWeb Reports & Documents
18. Module A2 — OWL Frontend Framework
19. Module A3 — POS Customization (OWL)
20. Module A4 — External APIs (XML-RPC)
21. Module A5 — Email Templates & Automation
22. Module A6 — Server Actions & Workflows

Part E — DevOps & Production

23. Module D1 — Docker & Containerization
24. Module D2 — VPS, Nginx & SSL Deployment
25. Module D3 — Performance & Worker Tuning
26. Module D4 — CI/CD & Monitoring Basics

Part F — Delivery Framework

27. Recommended Weekly Schedule
28. Capstone Project Briefs
29. Instructor Delivery Checklist
30. Appendices — Cheat Sheets & Glossary

01 Executive Summary & Program Vision

This Training Plan defines a complete, production-oriented pathway for becoming a confident Odoo Full-Stack Developer. It unifies operating-system fluency, Python mastery, PostgreSQL administration, Odoo backend/ORM development, modern OWL frontend work, Point of Sale customization, integration APIs, and real-world DevOps deployment.

The curriculum is intentionally sequenced: learners first earn the platform skills that Odoo depends on (Linux, Python, PostgreSQL), then progress through Odoo 17 development fundamentals, continue into Odoo 18/19 security and privilege model changes, and finish with containerization, reverse-proxy hardening, and performance architecture.

Program Pillars

Pillar	Focus	Outcome
Foundations	Linux · Python · PostgreSQL	Reliable server & coding baseline
Odoo Backend	Models · Fields · ORM · Views · Security	Ship maintainable business modules
Odoo Frontend	OWL · Widgets · POS · RPC	Extend the web client professionally
Integration	XML-RPC · Email · Server Actions	Connect Odoo to external systems
Operations	Docker · Nginx · SSL · Workers	Deploy and tune production instances

North-Star Outcome: Graduates can install Odoo, develop custom modules end-to-end, secure data with groups and record rules, customize OWL/POS, expose integrations, and operate a hardened VPS deployment.

02 Who This Program Is For

Primary Audiences

- Junior developers transitioning into Odoo customization and module development
- ERP consultants who need stronger technical implementation skills
- Python developers entering the Odoo ecosystem (17 → 19)
- System administrators expanding into application-layer Odoo work
- Technical leads designing internal Odoo enablement academies

Prerequisites

Level	Requirement	Notes
Required	Basic computer literacy	Comfort with files, browsers, accounts
Recommended	Any prior programming exposure	Python preferred but not mandatory
Recommended	Familiarity with Ubuntu desktop/server	Covered in Module F1 if missing
Helpful	Business process awareness	Sales, inventory, HR, accounting concepts

Competency Targets (Bloom Levels)

- Remember & Understand — CLI, ORM vocabulary, security components, view types
- Apply — Scaffold modules, define fields, build views, write record rules
- Analyze — Diagnose worker limits, slow SQL, inheritance conflicts, ACL gaps
- Create — Deliver a multi-module business app with reports, security, and deployment

03 Learning Journey Map

The journey is organized as six progressive phases. Each phase unlocks the next; skipping foundations is discouraged because Odoo debugging frequently requires Linux, Python, and PostgreSQL literacy.

Phase	Modules	Theme	Exit Gate
1 Foundations	F1–F3	OS, language, database	CLI + Python + SQL labs passed
2 Odoo Core	O1–O4	Install, modules, fields, ORM	Custom module with CRUD logic
3 UX & Security	O5–O8	Views, ACLs, data files	Secured app with list/form/search
4 Advanced	A1–A6	QWeb, OWL, POS, APIs	Report + OWL/POS + XML-RPC demo
5 Ops	D1–D4	Docker, VPS, performance	HTTPS stack with workers tuned
6 Capstone	Projects	End-to-end delivery	Reviewed production-ready module

Version Coverage Strategy

Core development patterns are taught primarily on **Odoo 17** for stability and breadth of community tutorials, then explicitly mapped to **Odoo 18** and **Odoo 19** differences—especially the privilege model, attrs migration (invisible/readonly/required), and updated security practices. Installation labs include Ubuntu 22.04 (Odoo 17) and Ubuntu 24.04 (Odoo 19).

04 Program Structure & Duration Model

Two delivery models are supported. Instructors may compress or expand based on cohort seniority.

Model	Cadence	Total Hours	Best For
Intensive Bootcamp	Full-time, 6–8 weeks	140–160 hrs	Career switchers / dedicated cohorts
Professional Track	Part-time, 12–16 weeks	120–140 hrs	Working professionals

Hour Allocation by Domain

Domain	Guided Hours	Lab Hours	Share
Linux & CLI	8	6	≈10%
Python	16	12	≈20%
PostgreSQL	8	6	≈10%
Odoo Backend & Views	28	22	≈35%
OWL / POS / APIs / QWeb	14	12	≈18%
DevOps / Deploy / Perf	10	8	≈12%
Capstone & Review	6	10	≈10%

Note: Shares are approximate and intentionally overlap during integrated labs (for example, a security lab also reinforces views and ORM).

05 Assessment, Labs & Capstone Projects

Assessment Mix

Component	Weight	Evidence
Module Labs	40%	Working code pushed to learner repo + screenshots
Knowledge Checks	15%	Short quizzes on ORM, security, CLI, SQL
Integrated Mini-Projects	20%	Student Manager, Inventory tweak, POS button
Final Capstone	25%	Deployed module with docs, ACLs, report, README

Quality Rubric (Capstone)

- Architecture — clear models, sensible relations, no unnecessary complexity
- Correctness — CRUD, computed fields, constraints, and views behave as specified
- Security — groups, access rights, and record rules intentionally designed
- UX — list/form/search/kanban usable; decorations and statusbar where relevant
- Ops — installable via addons path; optional Docker Compose or systemd notes
- Documentation — manifest, README, and short demo script for reviewers

F1 Linux Operating System & CLI Mastery

Learning Outcome: Operate an Ubuntu server confidently: navigate the filesystem, manage users/services, inspect logs, secure SSH, and automate routine tasks with cron and apt.

Session Plan

Session 1 — OS Fundamentals & Navigation

- LINUX — OS Fundamentals & CLI Setup
- System Architecture & Kernel Basics
- Essential Terminal Navigation (pwd, cd, ls, tree)
- File Manipulation (cp, mv, rm, mkdir, touch)

Lab: Create a project tree under /opt/training and practice safe recursive operations.

Session 2 — Permissions, Users & Processes

- Linux File Permissions (chmod, chown, chgrp)
- Managing Users, Groups, and Sudo Privileges
- Process Management (top, htop, ps, kill, pkill)
- System V & Systemd Services (systemctl start/stop/status/enable)

Lab: Create an odoo system user concept dry-run; manage a sample systemd unit.

Session 3 — Logs, Networking, SSH & Automation

- Monitoring System Logs (journalctl, tail)
- Text Editors in CLI (Nano vs Vim)
- Networking Fundamentals (ip, ping, ss)
- SSH Key Generation, Configuration, and Hardening
- Environment Variables and .bashrc Configuration
- CRON Jobs and Task Automation
- Package Management with APT
- Disk Usage Monitoring (df, du, free)
- Linux Firewall Setup using UFW

Learning Outcome: Learner can harden SSH basics, enable UFW thoughtfully, and schedule a log-rotation-style cron.

F2 Python Programming for Odoo Developers

Learning Outcome: Write clean, idiomatic Python sufficient for Odoo models, ORM overrides, wizards, and small utilities—including OOP, errors, modules, and virtual environments.

Track Overview

This module follows a practical Python syllabus mapped to Odoo needs. Learners who are already strong in Python may place out via a diagnostic exam and join Odoo modules earlier.

Block	Topics	Odoo Relevance
Core Syntax	Interpreter, data types, numbers, strings, booleans, operators	Reading Odoo source & writing field logic
Collections	Lists, dicts, tuples, sets, unpacking, comprehensions	Recordsets, mapped/filtered patterns
Control Flow	Conditionals, loops, ternary, short-circuiting	Constraints, onchange-style logic, wizards
Functions	Args/kwargs, scope, decorators, walrus, clean code	Model methods, api decorators mindset
OOP	Classes, inheritance, MRO, dunder, polymorphism	Model inheritance & method overrides
FP Toolkit	map/filter/zip/reduce, lambdas, generators	Data transforms in reports & connectors
Tooling	venv, pip, PEP8, VS Code/PyCharm, debugging	Daily Odoo development workflow

Selected Lesson Sequence (Condensed)

- What is a programming language? · Python Interpreter · First program · Python 2 vs 3
- Variables, expressions, augmented assignment, strings, formatting, immutability
- Lists/matrices/methods · Dictionaries · Tuples · Sets · None
- Conditionals, truthy/falsey, logical operators, is vs ==
- for/while, range, enumerate, break/continue/pass · GUI mini-exercise
- Functions deep dive · Scope · global/nonlocal · Exam checkpoint
- OOP series through multiple inheritance & MRO
- Functional programming, decorators, errors, generators, modules/packages
- PyPI, pip, virtualenv · Useful stdlib · Debugging fundamentals

Lab Cap: Build a CLI contact manager using classes, file persistence, and exception handling.

F3 PostgreSQL Database Administration & Querying

Learning Outcome: Install and operate PostgreSQL for Odoo: users/roles, SQL fluency, indexes, EXPLAIN, backups, and configuration files used in real deployments.

Sessions & Topics

- Installing PostgreSQL on Ubuntu LTS · psql CLI · Users & roles · Grants
- PostgreSQL data types · DDL (CREATE/ALTER/DROP) · PK/FK/constraints
- CRUD · WHERE/LIKE/ILIKE/IN/BETWEEN · Joins (INNER/LEFT/RIGHT/FULL)
- Aggregations (GROUP BY/HAVING/COUNT/SUM/AVG)
- Indexes & optimization (B-Tree, GIN, composite) · EXPLAIN ANALYZE
- Backup/export with pg_dump · Restore with pg_restore/psql
- postgresql.conf & pg_hba.conf · Locks, connections, terminating stuck queries

Learning Outcome: Learner can diagnose slow queries and safely back up/restore an Odoo-like database.

Lab: Create an odoo DB role, sample schema, indexed queries, and a dump/restore drill.

Odoo-Specific SQL Habits

- Always prefer ORM for business logic; use SQL for diagnosis and DBA tasks
- Inspect `ir_model_data` / `ir_model_access` when debugging security & XML IDs
- Use `pg_stat_statements` later in performance module for production slow-query analysis

O1 Odoo Installation, CLI & Configuration

Learning Outcome: Install Odoo 17 on Ubuntu 22.04 (and preview Odoo 19 on Ubuntu 24.04), start odoo-bin, author a configuration file, and operate core CLI workflows safely.

Tutorial Units Covered

- Odoo 17 Installation on Ubuntu 22.04 LTS Tutorial
- How to start Odoo from terminal | odoo-bin
- Introduction to Odoo 17 CLI Commands
- How to Create a Configuration File for Odoo 17
- Exploring Common URLs in Odoo Database Manager
- How to Disable Database Manager / Selector URL (Security)
- Mastering Odoo 17 Database CLI Commands
- Configuring Odoo 17 in PyCharm Tutorial
- Basic CLI Commands · Install/Upgrade Modules via CLI
- SHELL CLI · Master password reset · Prevent HTTP call in CLI
- Odoo Useful URLs · Log files · Custom logs · Logger CLI (Web/SQL logs)
- Upgrade Code CLI Command | Module Migration
- Odoo 19 Installation on Ubuntu 24.04 LTS (Latest track)

Hands-on Checklist

Task	Acceptance Criteria
Install dependencies & Odoo source	odoo-bin --help works; server starts on 8069
Create odoo.conf	db_host/user/password, addons_path, logfile set
Secure DB manager	list_db=False validated; master password documented
IDE run config	PyCharm/VS Code launches with same conf
Logging	Custom logger writes to file; SQL debug toggled intentionally

O2 Custom Modules, Manifest & Addons Paths

Learning Outcome: Scaffold a module, configure manifests and dependencies, manage multi-addons paths (including App Store layouts), and auto-install modules in controlled environments.

- Create module using SCAFFOLD command (Odoo 17)
- Adding Custom Paths · Odoo App Store · Multi Addons Path
- Understanding the manifest File in Odoo 17
- Adding Dependencies in Odoo 17 Manifest File
- How to create a custom module in Odoo 17
- Automatically Installing Modules in Odoo 17
- How `_name` and `_table` model attributes work
- Auto Create Fields In Model
- How to add multiple addons using addons-path

Manifest Quality Standards

- name, version, category, summary, description, author, license filled deliberately
- depends lists only required modules; avoid hidden transitive assumptions
- data/demo XML ordered to respect foreign-key and XML-ID dependencies
- installable/application flags match whether the module is an app or technical library

Lab: Scaffold `wl_student`, wire `addons_path`, install via UI and CLI.

O3 Fields & Model Design

Learning Outcome: Select the correct field types for business data—including relational, computed, monetary, binary/image—and understand when related/reference fields are appropriate.

Field Catalog (Teach + Practice Each)

Category	Fields
Scalar	Char · Text/Textarea · Html · Boolean · Selection · Integer · Float · Json
Temporal	Date · Datetime
Relational	Many2one · One2many · Many2many · Reference · Related
Advanced	Compute · Monetary · Binary (upload) · Image

Design Guidelines

- Prefer Many2one + One2many over duplicated Char references
- Use Selection for closed vocabularies; Many2one when users must extend values
- Monetary fields require currency_id discipline
- Computed fields: define depends carefully; decide store=True only when needed
- Binary/Image: consider attachment storage and access rules

Lab: Extend student module with personal data, enrollment lines (O2M), tags (M2M), photo, computed age, and monetary fee fields.

O4 ORM Methods Deep Dive

Learning Outcome: Master default ORM methods, overrides (create/write/copy/unlink), search domains, recordset helpers, and modern fetch APIs used in Odoo 17+.

A. Lifecycle Overrides

- Understanding Default ORM Methods in Odoo
- Override create · write · copy (duplicate) · unlink (remove)
- Add a Custom Method · exists() to verify records

B. Query & Read APIs

- search() · search domain operators · search_count()
- read · read_group · search_read · search_fetch
- name_create · default_get · _name_search · get_view() (fields_view_get legacy map)
- fields_get · get_metadata

C. Recordset Helpers

- ensure_one() · filtered() · mapped() · sorted() · grouped()

D. Environment & Special Commands

- Odoo Environment / self.env · Modify environment variables patterns
- Special commands overview and practice: [0,0,vals] create · [1,id,vals] update · [2,id] delete · [3,id] unlink · [4,id] link · [5] clear · [6,0,ids] set
- Active field · Archive/Unarchive · Soft vs Hard Delete

Safety Rule: Never bypass ACLs casually with sudo(); document every elevated operation and prefer explicit security design.

O5 Views — List, Form, Search, Kanban & More

Learning Outcome: Build polished business UX with list/form/search/kanban/graph/pivot/cohort/gantt patterns and modern Odoo 17+ view attributes.

List View Mastery

- Create list view · Editable list · Mass editing · Decorations
- Button to open form · Sort By & Limit · Hide/show fields · Sum/Average
- Field width / hide label · Group By · Headers & buttons
- Field-level decoration · Disable buttons · Drag-drop handle widget

Search & Form

- Create Search View · Search Panel · Dynamic Datetime Filter · Default Filter
- Form customization · Sheet/Header/Group/Newline/Separator tags
- Notebook pages · Button containers · Statusbar · Dynamic statusbar

Kanban & Analytical Views

- Kanban create · Drag & drop · Images on cards · Progressbar widget
- Graph (bar/pie/line) · Pivot · Cohort · Gantt

View Attribute Migration Tips (Odoo 17+)

- What is attrs in views — migration away from legacy attrs dictionaries
- Readonly · Invisible · Required attributes in views (modern expressions)

O6 Security Architecture (Groups, ACLs, Rules)

Learning Outcome: Design least-privilege security using categories, groups, access rights, record rules, and Odoo 19 privilege-model awareness.

Six Essential Components (Framework Lecture)

Deliver the complete security guide covering users, groups, access rights, record rules, field/view restrictions, and menu/action visibility—then implement each on the student module.

Unit	Learning Focus
Groups & Categories	Create groups · parent/child · custom category · combobox style
Implied Groups	implied_ids inheritance patterns with real examples
Access Rights	CSV and XML definitions of ir.model.access
Record Rules	Why required · student/teacher/admin scenario · inheritance
Application Points	Views, fields, window actions, menus
Odoo 19 Delta	New privilege model · migration notes · groups on fields/views/actions

Lab: Implement Admin / Teacher / Student roles with record rules isolating private notes while allowing shared course visibility.

O7 Data Loading — XML & CSV

Learning Outcome: Load deterministic demo/production data via XML and CSV, including relational fields, binaries/images, and eval attributes.

- Intro: record creates using XML or CSV
- Create records with XML · eval attribute · add images
- Many2one / Many2many / One2many in XML
- Binary fields in XML
- CSV records · Many2one in CSV · Many2many in CSV · images/binary in CSV
- Difference between XML and CSV data import

Instructor Emphasis

- XML IDs are API contracts—name them stably
- Use noupdate carefully for editable master data vs locked system data
- Prefer XML for security/view/action definitions; CSV shines for bulk ACL rows and simple masters

O8 View Inheritance & Extension Patterns

Learning Outcome: Extend existing Odoo views safely using inheritance, xpath, and position strategies without breaking upgrades.

- View Inheritance overview — extend views basics
- Inheritance using field tag
- Inheritance using xpath
- Positions: after · before · inside · replace · attributes

Upgrade-Safe Practices

- Prefer stable name/attrs anchors over brittle absolute xpaths when possible
- Keep inherited diffs small and documented
- Test against base module upgrades in a staging database

Also covered in this security/UX cluster: URL actions (internal/external), ir.sequence auto-numbering patterns.

A1 QWeb Reports & Document Layout

Learning Outcome: Design printable PDF reports with QWeb: directives, control flow, inheritance, paper formats, barcodes/QR, and static/dynamic images.

- Introduction to QWeb report · Generate PDF reports
- t-field · t-esc · t-out usage
- Conditionals (if / elif / else) · Loops · predefined loop variables
- Define variables · Call templates · t-att · t-attf
- Inherit/extend QWeb templates
- Paper format / document layout · Custom report end-to-end
- Barcode & QR generation · Static images · Dynamic images

Lab: Student ID card / enrollment confirmation PDF with logo, QR, and conditional watermark.

A2 Odoo Web Library (OWL) Framework

Learning Outcome: Build and extend reactive OWL components, templates, field widgets, and web client integrations using rpc services and the patch utility.

1) Architecture & Components

- Introduction to OWL & reactive architecture
- Component structure & lifecycle hooks
- State management with useState and useRef
- Event handling and props passing

2) UI Templates & Custom Widgets

- OWL QWeb XML templates & directives (t-if, t-foreach, t-model)
- Custom form field widgets (Odoo 17/18)
- Overriding existing JS components & views
- Patching standard OWL components with patch()

3) Service Layer & WebClient

- Asynchronous RPC via Odoo rpc service
- Building custom views (Kanban/Dashboard) with OWL

A3 OWL POS Customization

Learning Outcome: Customize Point of Sale screens with OWL: buttons, click handlers, popups, translations, RPC data rendering, and order-line utilities.

- Add custom button in POS (Odoo 16+ OWL patterns applicable forward)
- Create click events for POS buttons
- Hide buttons on POS screen
- Create a Popup in Odoo POS
- Translate strings in JavaScript files
- JavaScript RPC calls in POS · Render dynamic data via RPC
- Clear all order lines in POS

Lab: Add a “Manager Note” button opening a popup, storing a note via RPC, with translated labels.

A4 External API — XML-RPC Integration

Learning Outcome: Authenticate and perform CRUD against Odoo from external systems using XML-RPC, including Postman-based verification workflows.

- XML-RPC Integration Tutorial — connect external systems
- External API Authentication — secure login patterns
- Fetching/exporting data via XML-RPC
- CRUD operations: add, update, remove records
- Authentication in Postman · Add/update/delete via Postman

Security & Design Notes

- Use dedicated integration users with minimal ACLs
- Prefer HTTPS endpoints; never embed production passwords in notebooks
- Idempotent external writes: plan XML IDs or external_id mapping strategies

A5–A6 Email Templates, Automation & Server Actions

Email

- How to create an Email Template
- How to send email from Odoo using an Email Template

Server Actions

- What is a Server Action in Odoo? Full explanation with use cases
- Connect server actions to UI buttons and automated workflows
- Safe coding practices inside server action Python code

Integration Lab: Combine record rules + server actions + email templates to implement an enrollment approval notification flow without hard-coding UI-only logic.

D1 Docker Containerization & Compose

Learning Outcome: Containerize Odoo development and staging stacks with Docker Engine, Compose, volumes, networking, and safe upgrade practices.

- Containerization vs Virtualization
- Install Docker Engine & Docker Compose on Ubuntu
- Essential CLI: `docker run/ps/exec/logs/rm`
- Images, containers, Docker Hub
- Custom Dockerfiles for Odoo development
- Volumes for Postgres & addons persistence
- Docker networking · Compose YAML structure
- Multi-container environment (Odoo + PostgreSQL)
- `.env` files · Updating/upgrading without data loss
- Basic CI/CD concepts for Odoo modules (GitHub Actions)

Lab: Compose file with `odoo+db`, mounted custom addons, named volumes, and documented upgrade steps.

D2 Production VPS, Nginx & SSL

Learning Outcome: Deploy Odoo on a VPS with hardened SSH, disciplined directory layout, Nginx reverse proxy, longpolling, Let's Encrypt SSL, systemd, and backup cron jobs.

Layer	Topic / Tutorial Unit
VPS Provisioning	Choosing VPS specs (AWS, DigitalOcean, Hetzner) & initial setup
Server Security	Hardening SSH — disable password auth & change default port
Directory Layout	/opt/odoo · /var/log/odoo · custom addons layout
Reverse Proxy	Install/configure Nginx as reverse proxy
Port Routing	Forward 80/443 → Odoo 8069
WebSockets	Nginx for Gevent/Longpolling (8072)
SSL Security	Let's Encrypt + Certbot automation
Systemd Service	odoo.service setup
Backups	DB + filestore backup scripts via cron

D3 Performance Tuning & Worker Architecture

Learning Outcome: Size workers correctly, set memory/time limits, tune PostgreSQL connections, and introduce caching/monitoring patterns for scale.

- Odoo multiprocessing & worker architecture
- Calculating optimal workers from CPU cores and RAM
- `limit_memory_hard` / `limit_memory_soft`
- `limit_time_cpu` / `limit_time_real`
- PostgreSQL connection pooling & `max_connections`
- Longpolling / WebSocket handlers for scale
- Spotting missing indexes on custom models
- Slow queries with `pg_stat_statements`
- Redis for session storage/caching (distributed setups)
- Load balancing multiple Odoo instances behind Nginx
- Monitoring with Prometheus and Grafana (overview)

Lab: Produce a capacity worksheet for a 4-vCPU / 8 GB VPS and apply conf changes with before/after notes.

D4 CI/CD & Monitoring Basics

Close the operations loop with lightweight automation: lint/test on pull requests, build checks for modules, and basic uptime/resource monitoring.

- GitHub Actions workflow sketch for module CI (install, pylint-odoo optional, tests)
- Environment secrets management principles
- Smoke-test checklist after deploy (login, ACL sample, report print, longpolling)
- Prometheus/Grafana dashboards — what to watch first (CPU, RAM, DB connections, response time)

27 Recommended Weekly Schedule

Example 12-week Professional Track (≈10–12 hours/week). Intensive cohorts can map each week below to ~2–3 bootcamp days.

Week	Focus	Deliverable
1	Linux CLI + SSH + systemd	Hardened practice VM notes
2	Python core + collections + flow	Exercises set A
3	Python OOP + modules + venv	CLI app + diagnostic quiz
4	PostgreSQL + backups	SQL lab workbook
5	Odoo install + conf + CLI	Running local Odoo 17
6	Modules + fields	wl_student scaffold with field set
7	ORM methods + special commands	Overrides + domain drills
8	Views + inheritance attributes	List/form/search/kanban pack
9	Security groups/ACLs/rules (+ Odoo 19 notes)	Role-based demo DB
10	QWeb + XML/CSV data	PDF report + demo data
11	OWL/POS + XML-RPC + email/actions	Integration mini-project
12	Docker + VPS/Nginx/SSL + workers + capstone	Deployed capstone defense

28 Capstone Project Briefs

Capstone A — Academic Institute Manager

- Models: Student, Course, Enrollment, Attendance, Fee Payment
- Views: list/form/search/kanban/graph; statusbar on enrollment
- Security: Admin/Registrar/Teacher groups with record rules
- Report: Enrollment confirmation PDF with QR
- Automation: Email on enrollment confirmation (template)
- Ops bonus: Docker Compose or Nginx deployment notes

Capstone B — Retail POS Helper

- POS button + popup + RPC persistence of cashier note
- Backend model storing notes linked to pos.order
- ACL for managers only on note analysis menu
- Optional XML-RPC script to export daily notes

Capstone C — Ops Excellence Pack

- Production-like directory layout + systemd unit
- Nginx + longpolling + Certbot (or staging TLS)
- Backup cron + restore rehearsal document
- Worker sizing worksheet + pg_stat_statements sample

29 Instructor Delivery Checklist

Before Cohort Start

- Provision shared lab VM image (Ubuntu 22.04) and optional 24.04 preview image
- Prepare Odoo 17 source pin + sample odoo.conf + addons skeleton
- Validate PostgreSQL role scripts and Docker Compose demo
- Distribute this Training Plan + Presentation PDF

During Delivery

- Start each module with outcomes and exit gate
- Live-code then freeze a reference commit for learners
- Enforce security discussions before fancy UX
- Keep a parking lot for Odoo 19 deltas; schedule explicit comparison session

After Delivery

- Collect rubrics and archive capstone repos
- Update curriculum changelog (attrs, privilege model, CLI flags)
- Retrospective: which labs overran; adjust hour table

30 Appendices — Cheat Sheets & Glossary

Appendix A — Essential Odoo CLI Flags

Flag / Command	Purpose
odoo-bin -c odoo.conf	Start with configuration file
-i module_name	Install module
-u module_name	Upgrade module
--db-filter=^db\$	Restrict DB listing/selection
shell -d dbname	Interactive ORM shell
--stop-after-init	Run init/ops then exit (CI friendly)

Appendix B — Security Object Quick Map

Object	Role
res.groups	Permission bundles assigned to users
ir.module.category	Security category / app grouping
ir.model.access	CRUD permissions per model/group
ir.rule	Record-level domain restrictions
groups on views/fields	UI and field visibility controls

Appendix C — Special Commands Reminder

Command	Meaning
(0, 0, values)	Create a new record on O2M/M2M
(1, id, values)	Update existing linked record
(2, id)	Remove and delete record
(3, id)	Unlink / remove relation without deleting
(4, id)	Link existing record
(5,)	Unlink all (clear)
(6, 0, ids)	Replace set with given ids

Appendix D — Glossary (Selected)

- **Addon / Module** — installable unit with manifest and code
- **Recordset** — ORM collection of records supporting mapped/filtered/sorted
- **XML ID** — external identifier in `ir.model_data`
- **Longpolling** — bus/websocket-style notifications channel (8072)
- **Worker** — multiprocess HTTP worker serving concurrent requests
- **OWL** — Odoo Web Library reactive UI framework
- **QWeb** — templating engine for reports and web client XML
- **Privilege (Odoo 19)** — evolved security concept beyond classic-only groups

Curriculum Closure & Next Steps

This Training Plan is the master syllabus for the Odoo Full-Stack Developer Program. Use it with the companion Presentation PDF for kickoffs, stakeholder reviews, and week-one orientation. Instructors should treat version deltas (especially Odoo 19 security) as living addenda.

Recommended Companion Assets

- Odoo_FullStack_Training_Presentation.pdf — slide deck for facilitation
- Lab repository template with wl_student skeleton
- Docker Compose starter + Nginx conf samples
- Capstone rubric spreadsheet

Program Principle: Ship competence, not just content coverage. Every module ends with a working artifact that could survive a code review in a real Odoo project.

Document control

Field	Value
Document Title	Odoo Full-Stack Developer Training Plan
Edition	2026.07
Primary Versions	Odoo 17 (core) · Odoo 18/19 (delta track)
Classification	Training Curriculum — Professional
Page Target	30+ pages (this edition)